MOON

Programming Language Specification

Codename: Mars

Version 0.1.0 • Draft Specification
February 2026

CONFIDENTIAL
Google • Chrome Platform Team

Table of Contents

1. Executive Summary
2. Design Philosophy
3. Language Overview
4. Type System
5. Memory Model
6. Concurrency
7. UI Framework (MoonView)
8. Platform Capability API
9. Hardware Security Bridge
10. Package Manager (Aurora)
11. Standard Library
12. Interoperability
13. Compilation Pipeline
14. Migration Strategy
15. Appendix: Grammar Reference

1. Executive Summary

Moon is a statically-typed, compiled programming language designed to replace JavaScript and the DOM as the foundational technology of the web platform. Developed under codename Mars, Moon targets a new browser runtime that eliminates the legacy constraints of HTML, CSS, and JavaScript while providing native-quality experiences across all platforms.

Moon is not an incremental improvement. It is a ground-up replacement for the web's execution model, designed with the following non-negotiable requirements:

- **Type safety:** All types are known at compile time. No any, no dynamic, no runtime type coercion.
- **Memory safety:** Automatic Reference Counting with cycle detection. No garbage collection pauses. No manual memory management.
- **Compiled execution:** AOT compilation for production. Interpreted mode for development with hot reload.
- **No DOM:** Declarative, reactive UI scene graph replaces the Document Object Model entirely.
- **No CSS:** Type-safe styling properties on view components. No cascade, no specificity, no selectors.
- **Platform-aware:** The runtime detects the underlying platform and bridges to native capabilities (Apple Pay, Google Pay, APNs, FCM, biometrics, hardware security modules).
- **Hardware security:** First-class support for hardware security modules (HSMs), secure enclaves, and hardware tokens for cryptographic operations.
- **Engine-agnostic:** On Apple hardware, the runtime operates on top of WebKit. On other platforms, it uses Blink. The developer never knows or cares.

Moon's syntax is inspired by Kotlin, prioritizing readability, conciseness, and developer ergonomics. Its memory model borrows from Swift's ARC with an additional lightweight cycle detector. Its UI framework draws from SwiftUI, Jetpack Compose, and Flutter's declarative paradigms.

2. Design Philosophy

2.1 Guiding Principles

Declare intent, not mechanics. Developers describe what they want, not how to achieve it. The runtime and compiler handle platform-specific implementation.

One codebase, native everywhere. A Moon application runs identically on macOS (via WebKit), Android (via Blink), Windows (via Blink), and Linux (via Blink), with each platform delivering its best native experience.

Fail at compile time, not runtime. If code compiles, it should work. The type system, null safety, exhaustive pattern matching, and capability declarations catch errors before deployment.

Security is not optional. Hardware-backed cryptography, signed packages, capability-based permissions, and zero-trust networking are built into the language and runtime, not bolted on as libraries.

Performance is a feature. ARC memory management, AOT compilation, and a retained-mode renderer ensure consistent 60/120fps UI performance without garbage collection pauses.

2.2 What Moon Replaces

Legacy Technology	Moon Replacement	Why
JavaScript / TypeScript	Moon language	Type-safe, compiled, memory-safe
HTML DOM	MoonView scene graph	Declarative, reactive, retained-mode
CSS	Inline type-safe styling	No cascade, no specificity wars
npm / node_modules	Aurora package manager	Signed binaries, flat deps, no install scripts
WebAuthn (auth only)	Hardware Crypto Bridge	Full HSM operations, not just auth
Web Crypto API (software)	Hardware-backed crypto	Keys never leave hardware
Service Workers	Native push integration	APNs/FCM/WNS via platform bridge
Payment Request API	Native payment bridge	Apple Pay/Google Pay/native wallets

3. Language Overview

3.1 Basic Syntax

Moon uses Kotlin-inspired syntax with significant enhancements for web platform capabilities. Every Moon source file uses the .moon extension.

3.1.1 Hello World

```
// main.moon
fun main() {
    print("Hello, Moon!")
}
```

3.1.2 Variables and Constants

```
val name: String = "Moon"     // immutable (preferred)
var counter: Int = 0           // mutable
val inferred = 42              // type inference
val pi = 3.14159              // Float64 by default

// Null safety (no billion-dollar mistake)
val required: String = "always set"
val optional: String? = null   // explicitly nullable
val length = optional?.length ?: 0  // safe call + elvis
```

3.1.3 Functions

```
fun add(a: Int, b: Int): Int {
    return a + b
}

// Expression body
fun multiply(a: Int, b: Int): Int = a * b

// Default parameters
fun greet(name: String, greeting: String = "Hello"): String {
    return "${greeting}, ${name}!"
}

// Lambda / closure
val doubled = listOf(1, 2, 3).map { it * 2 }

// Trailing lambda
items.filter { it.price > 10.0 }
      .sortedBy { it.name }
```

```
        .take(5)
```

3.1.4 Control Flow

```
// when expression (exhaustive, replaces switch)
val result = when (status) {
    Status.Active -> "Running"
    Status.Paused -> "On hold"
    Status.Stopped -> "Done"
    // No default needed - compiler enforces exhaustiveness
}

// if as expression
val label = if (count > 0) "Items: $count" else "Empty"

// for loops
for (item in cart.items) {
    print(item.name)
}

// Range
for (i in 0 until 10) { /* 0..9 */ }
for (i in 10 downTo 0 step 2) { /* 10, 8, 6... */ }
```

3.2 Classes and Data Types

3.2.1 Data Classes

```
// Immutable data with auto-generated equals, hashCode, copy
data class User(
    val id: String,
    val name: String,
    val email: String,
    val role: Role = Role.Viewer
)

// Destructuring
val (id, name, email) = user

// Copy with modifications
val admin = user.copy(role = Role.Admin)
```

3.2.2 Sealed Classes (Algebraic Data Types)

```
sealed class Result<out T> {
    data class Success<T>(val data: T) : Result<T>()
    data class Error(val message: String,
                     val code: Int) : Result<Nothing>()
    object Loading : Result<Nothing>()
}

// Exhaustive matching
fun handleResult(result: Result<User>) = when (result) {
    is Result.Success -> showUser(result.data)
    is Result.Error -> showError(result.message)
    Result.Loading -> showSpinner()
}
```

3.2.3 Interfaces and Traits

```
interface Serializable {
    fun toBytes(): ByteArray
    fun contentType(): String = "application/octet-stream"
}

interface Cacheable {
    val cacheKey: String
    val ttl: Duration get() = 5.minutes
}
```

```
// Multiple interface implementation
class ApiResponse(
    override val cacheKey: String,
    val body: ByteArray
) : Serializable, Cacheable {
    override fun toBytes() = body
}
```

3.2.4 Enums

```
enum class HttpMethod {
    GET, POST, PUT, DELETE, PATCH;

    val isIdempotent: Boolean
        get() = this in listOf(GET, PUT, DELETE)
}

enum class Priority(val level: Int) {
    Critical(0),
    High(1),
    Medium(2),
    Low(3)
}
```

4. Type System

Moon's type system is designed to eliminate entire categories of runtime errors. Every value has a known type at compile time. There is no any type, no implicit coercion, and no escape hatches.

4.1 Primitive Types

Type	Size	Description
Int	64-bit	Signed integer (default numeric type)
Int8, Int16, Int32	8/16/32-bit	Sized integers
UInt, UInt8, UInt16, UInt32, UInt64	Various	Unsigned integers
Float32	32-bit	Single precision IEEE 754
Float64	64-bit	Double precision (default float)
Bool	1-bit	true or false
Char	32-bit	Unicode scalar value
String	Variable	UTF-8 encoded, immutable
Byte	8-bit	Alias for UInt8
Unit	0-bit	Equivalent to void/Void
Nothing	0-bit	Bottom type (function never returns)

4.2 Null Safety

Null reference errors are eliminated at compile time. Types are non-nullable by default. Nullable types must be explicitly declared with the ? suffix.

```
val name: String = "Moon"      // cannot be null, ever
val alias: String? = null       // explicitly nullable

// Compiler error: type mismatch
// val bad: String = null        // COMPILE ERROR

// Safe operations on nullable types
val len: Int? = alias?.length           // safe call
val lenOrZero: Int = alias?.length ?: 0 // elvis operator
val lenForSure: Int = alias!!.length    // assert non-null (throws if null)

// Smart casting after null check
if (alias != null) {
```

```
    // alias is automatically String (not String?) here
    print(alias.length)
}
```

4.3 Generics

```
// Generic class with variance
class Repository<T : Entity> {
    private val items = mutableListOf<T>()

    fun add(item: T) { items.add(item) }
    fun find(id: String): T? = items.firstOrNull { it.id == id }
    fun all(): List<T> = items.toList()
}

// Generic function with constraints
fun <T> merge(a: List<T>, b: List<T>): List<T>
    where T : Comparable<T>,
          T : Serializable {
    return (a + b).sorted()
}

// Covariance and contravariance
interface Producer<out T> { fun produce(): T }
interface Consumer<in T> { fun consume(item: T) }
```

4.4 Type Aliases and Extensions

```
// Type aliases for clarity
typealias UserId = String
typealias Handler<T> = (T) -> Unit
typealias JSON = Map<String, Any>

// Extension functions
fun String.isValidEmail(): Bool {
    return this.matches(Regex("[a-zA-Z0-9+._-]+@[a-zA-Z0-9.-]+"))
}

// Extension properties
val List<Int>.median: Float64
    get() {
        val sorted = this.sorted()
        return if (size % 2 == 0)
            (sorted[size/2 - 1] + sorted[size/2]) / 2.0
        else sorted[size/2].toFloat64()
    }
```

5. Memory Model

Moon uses Automatic Reference Counting (ARC) as its primary memory management strategy, supplemented by a lightweight cycle detector. This provides deterministic memory deallocation without garbage collection pauses, ensuring smooth UI rendering at 60/120fps.

5.1 ARC Semantics

```
// Reference counting is automatic and invisible
fun processOrder(cart: Cart) {
    val order = Order(cart)  // refcount = 1
    val receipt = order.generateReceipt()  // order refcount = 2
    sendEmail(receipt)
    // receipt goes out of scope: order refcount = 1
    // order goes out of scope: refcount = 0, deallocated immediately
}
```

5.2 Weak and Unowned References

To prevent reference cycles, Moon provides weak and unowned reference qualifiers.

```
class Parent {
    var children: List<Child> = emptyList()
}

class Child {
    // weak breaks the cycle - becomes null when Parent is deallocated
    weak var parent: Parent? = null
}

class Engine {
    // unowned: guaranteed non-null, crashes if accessed after dealloc
    unowned val vehicle: Vehicle
}
```

5.3 Cycle Detector

Moon’s runtime includes a lightweight background cycle detector that identifies and breaks reference cycles the developer may have missed. Unlike a full garbage collector, it runs incrementally on a low-priority thread and only scans objects that the compiler cannot prove cycle-free at compile time.

```
// The compiler statically analyzes reference graphs
// Objects with no possible cycles are marked @NoCycle
// Only potential-cycle objects are registered with the detector

@NoCycle  // compiler-inferred, not written by developer
```

```
data class Point(val x: Float64, val y: Float64)

// Classes with bidirectional references are cycle-candidates
class TreeNode {
    var children: List<TreeNode> = emptyList()
    weak var parent: TreeNode? = null  // weak breaks the cycle
}
```

6. Concurrency

Moon provides structured concurrency inspired by Kotlin coroutines and Swift's async/await, with compile-time data race safety.

6.1 Async/Await

```
// Suspending functions
suspend fun fetchUser(id: String): User {
    val response = http.get("https://api.example.com/users/$id")
    return response.decode<User>()
}

// Calling suspend functions
suspend fun loadProfile() {
    val user = fetchUser("123")      // suspends, doesn't block
    val posts = fetchPosts(user.id)   // sequential
    render(ProfileView(user, posts))
}
```

6.2 Structured Concurrency

```
suspend fun loadDashboard() {
    // Parallel execution within a scope
    val (user, orders, notifications) = concurrent {
        val user = async { fetchUser(userId) }
        val orders = async { fetchOrders(userId) }
        val notifications = async { fetchNotifications(userId) }
        Triple(user.await(), orders.await(), notifications.await())
    }
    // All three complete before we continue
    render(Dashboard(user, orders, notifications))
}
```

6.3 Actors (Thread Safety)

```
// Actors provide thread-safe mutable state
actor ShoppingCart {
    private var items: MutableList<CartItem> = mutableListOf()

    fun add(item: CartItem) {
        items.add(item)
    }

    fun total(): Float64 {
```

```
        return items.sumOf { it.price * it.quantity }
    }

    fun checkout(): Order {
        val order = Order(items.toList())
        items.clear()
        return order
    }
}

// All access is automatically serialized
suspend fun addToCart(cart: ShoppingCart, item: CartItem) {
    cart.add(item)  // safe from any thread/coroutine
    val total = cart.total()
    updateUI(total)
}
```

6.4 Reactive Streams

```
// Built-in reactive stream type
val priceStream: Flow<Float64> = stockTicker("GOOG")

// Transform and collect
priceStream
    .filter { it > 150.0 }
    .map { PriceAlert(symbol = "GOOG", price = it) }
    .collect { alert ->
        notify(alert)
    }

// State flow for UI binding
val cartItems = MutableStateFlow<List<CartItem>>(emptyList())

// UI automatically re-renders when state changes
cartItems.value = cartItems.value + newItem
```

7. UI Framework (MoonView)

MoonView is Moon's declarative UI framework that replaces HTML, CSS, and the DOM. It uses a retained-mode scene graph with automatic diffing and efficient re-rendering. The programming model is inspired by SwiftUI and Jetpack Compose.

7.1 Basic Views

```
view ProductCard(product: Product) {
    Column(spacing = 12.dp, padding = 16.dp) {
        Image(product.thumbnailUrl)
            .size(width = 200.dp, height = 200.dp)
            .cornerRadius(8.dp)

        Text(product.name)
            .font(.headline)
            .color(.primary)

        Text("$${product.price}")
            .font(.body)
            .color(.secondary)

        Button("Add to Cart") {
            cart.add(product)
        }
        .style(.primary)
        .fullWidth()
    }
}
```

7.2 State Management

```
view Counter() {
    // @State triggers re-render on change
    @State var count = 0

    Column(alignment = .center, spacing = 16.dp) {
        Text("Count: $count")
            .font(.display)

        Row(spacing = 8.dp) {
            Button("-") { count-- }
                .style(.outlined)

            Button("+") { count++ }
```

```
                .style(.primary)
        }
    }
}
```

7.3 Navigation

```
// Declarative navigation graph
navigation App {
    route("/") { HomePage() }
    route("/products") { ProductList() }
    route("/products/{id}") { params ->
        ProductDetail(productId = params["id"])
    }
    route("/cart") { CartPage() }
    route("/checkout") { CheckoutFlow() }
}

// Programmatic navigation
Navigator.push("/products/${product.id}")
Navigator.pop()
Navigator.replace("/checkout")
```

7.4 Styling System

Moon eliminates CSS entirely. Styling is type-safe, scoped to components, and composable.

```
// Reusable style definitions
val cardStyle = Style {
    background(.surface)
    cornerRadius(12.dp)
    shadow(elevation = 4.dp, color = .black.opacity(0.1))
    padding(16.dp)
}

// Apply styles
Column {
    Text("Hello")
}.applyStyle(cardStyle)

// Responsive layout
view ResponsiveGrid(items: List<Product>) {
    Grid(
        columns = when (Screen.width) {
            in 0.dp..600.dp -> 1
            in 600.dp..1200.dp -> 2
```

```
            else -> 4
        },
        spacing = 16.dp
    ) {
        for (item in items) {
            ProductCard(item)
        }
    }
}

// Theming
theme AppTheme {
    colors {
        primary = Color(0x5B21B6)
        secondary = Color(0x7C3AED)
        surface = Color(0xFFFFFF)
        background = Color(0xF9FAFB)
        error = Color(0xDC2626)
    }
    typography {
        display = Font("Inter", size = 36.sp, weight = .bold)
        headline = Font("Inter", size = 24.sp, weight = .semibold)
        body = Font("Inter", size = 16.sp, weight = .regular)
        caption = Font("Inter", size = 12.sp, weight = .regular)
    }
}
```

7.5 Animation

```
view AnimatedCard(isExpanded: Bool) {
    Column {
        Text("Details")
            .font(.headline)

        // Implicit animation
        if (isExpanded) {
            Text("Extended content here...")
                .transition(.slide + .opacity)
        }
    }
    .animate(.spring(damping = 0.8), on = isExpanded)
}

// Explicit animation control
@State var offset = 0.dp

Animation(.easeInOut, duration = 300.ms) {
```

```
    offset = 100.dp
}
```

8. Platform Capability API

Moon's runtime detects the underlying platform and bridges to native capabilities. Developers declare required capabilities in their manifest. The runtime resolves the optimal native implementation at launch.

8.1 Payments

```
@Capability(requires = Capability.Payments)
suspend fun checkout(cart: Cart): PaymentResult {
    val payment = Payment(
        amount = cart.total,
        currency = Currency.USD,
        items = cart.items.map { PaymentItem(it.name, it.price) }
    )

    // Runtime resolves:
    //   Apple device  -> Apple Pay (PassKit)
    //   Android/Chrome -> Google Pay
    //   Fallback       -> Stripe checkout sheet
    return Platform.payments.request(payment)
}
```

8.2 Notifications

```
@Capability(requires = Capability.Notifications)
suspend fun registerPush() {
    val token = Platform.notifications.register(
        permissions = listOf(
            NotificationPermission.Alert,
            NotificationPermission.Badge,
            NotificationPermission.Sound
        )
    )
    // Runtime resolves:
    //   Apple  -> APNs registration
    //   Android -> FCM token
    //   Windows -> WNS channel
    api.registerDevice(token)
}

@Notification(priority = .timeSensitive)
fun orderShipped(order: Order) {
    Platform.notifications.send(
        title = "Order Shipped!",
        body = "Your ${order.itemCount} items are on the way",
```

```
        data = mapOf("orderId" to order.id)
    )
}
```

8.3 Biometrics

```
@Capability(requires = Capability.Biometrics)
suspend fun authenticate(): AuthResult {
    return Platform.biometrics.authenticate(
        reason = "Confirm your identity",
        allowedMethods = listOf(
            BiometricMethod.FaceID,
            BiometricMethod.Fingerprint,
            BiometricMethod.DevicePasscode  // fallback
        )
    )
    // Runtime resolves:
    //   Apple  -> LocalAuthentication (Face ID / Touch ID)
    //   Android -> BiometricPrompt
    //   Windows -> Windows Hello
}
```

8.4 Platform Detection

```
// Automatic platform detection
val platform = Platform.current

when (platform.os) {
    OS.MacOS -> { /* WebKit runtime */ }
    OS.IOS -> { /* WebKit runtime */ }
    OS.Android -> { /* Blink runtime */ }
    OS.Windows -> { /* Blink runtime */ }
    OS.Linux -> { /* Blink runtime */ }
}

// Capability checking
if (Platform.supports(Capability.Payments)) {
    showPaymentButton()
} else {
    showManualCheckout()
}

// Engine detection
val engine = Platform.renderEngine  // WebKit | Blink
```

9. Hardware Security Bridge

Moon provides first-class support for hardware security modules (HSMs), secure enclaves, and hardware tokens. The browser runtime discovers available hardware and provides a unified cryptographic API. Private keys never leave the hardware device.

9.1 Hardware Discovery

```
@Capability(requires = Capability.HardwareCrypto)
suspend fun setupSecurity() {
    // Discover available hardware
    val devices = Platform.crypto.discoverDevices()

    // Returns list of available HSMs/tokens:
    // - YubiKey (USB/NFC)
    // - Apple Secure Enclave
    // - TPM 2.0
    // - Ledger (USB/BLE)
    // - Smart cards (PKCS#11)

    val preferred = devices.firstOrNull {
        it.supports(KeyType.Ed25519)
    } ?: throw SecurityException("No compatible hardware found")
}
```

9.2 Cryptographic Operations

```
// Sign data with hardware-backed key
suspend fun signDocument(document: ByteArray): Signature {
    val key = Platform.crypto.getKey(
        id = "document-signing-key",
        type = KeyType.Ed25519,
        requireHardware = true  // MUST be hardware-backed
    )

    // User authenticates via biometric/PIN
    // Hardware performs the signing
    // Only the signature is returned
    return key.sign(document)
}

// Encrypt for a recipient
suspend fun encryptMessage(
    message: ByteArray,
    recipientPublicKey: PublicKey
): EncryptedPayload {
```

```
    val myKey = Platform.crypto.getKey(
        id = "messaging-key",
        type = KeyType.X25519,
        requireHardware = true
    )

    return myKey.encrypt(
        plaintext = message,
        recipient = recipientPublicKey,
        algorithm = EncryptionAlgorithm.XChaCha20Poly1305
    )
}
```

9.3 Key Management

```
// Generate a new key pair on hardware
suspend fun generateIdentity(): PublicKey {
    val keyPair = Platform.crypto.generateKeyPair(
        id = "user-identity",
        type = KeyType.Ed25519,
        storage = KeyStorage.Hardware,
        authentication = KeyAuth.Biometric,
        exportable = false  // private key can NEVER be exported
    )

    // Only public key is accessible in code
    return keyPair.publicKey
}

// Key attestation - prove key is hardware-backed
suspend fun proveHardwareKey(key: CryptoKey): Attestation {
    return Platform.crypto.attest(key)
    // Returns signed attestation proving:
    //   - Key was generated on hardware
    //   - Key is non-exportable
    //   - Hardware model and firmware version
    //   - Attestation chain back to manufacturer
}
```

9.4 Capability Manifest

The application declares its cryptographic requirements in the Aurora manifest:

```
# aurora.manifest
crypto:
  operations: [sign, verify, encrypt, decrypt, keyExchange]
  key_types: [ed25519, x25519, ecdsa-p256]
```

```yaml
requires_hardware: true
fallback: none
attestation: required
devices:
  - fido2     # YubiKey, Titan
  - pkcs11    # Smart cards
  - secure_enclave  # Apple SE, TPM
  - bluetooth_hsm   # Ledger, custom BLE HSMs
```

10. Package Manager (Aurora)

Aurora is Moon's package manager, build system, and dependency resolver. It replaces npm, webpack, babel, eslint, and all associated configuration files with a single tool.

10.1 Project Structure

```
my-app/
  aurora.manifest        # project config + dependencies
  aurora.lock             # pinned dependency versions
  src/
    main.moon             # entry point
    views/
      HomePage.moon
      ProductList.moon
    models/
      User.moon
      Product.moon
    services/
      ApiClient.moon
      AuthService.moon
  tests/
    UserTest.moon
    ProductTest.moon
  assets/
    icons/
    fonts/
```

10.2 Manifest File

```
# aurora.manifest
name: my-app
version: 1.0.0
target: web
moon: 1.0

platform_capabilities:
  - payments
  - notifications
  - biometrics
  - crypto:
      requires_hardware: true
      operations: [sign, verify, encrypt, decrypt]

dependencies:
  moon-ui: 2.1.0
```

```
    moon-http: 1.4.0
    moon-json: 3.0.0
    moon-crypto: 1.0.0

  dev_dependencies:
    moon-test: 1.0.0
    moon-mock: 1.2.0
```

10.3 CLI Commands

Command	Description
aurora init <name>	Create new project with scaffolding
aurora add <package>	Add dependency (exact version pinned)
aurora remove <package>	Remove dependency
aurora build	Compile to runtime bytecode (production)
aurora dev	Start dev server with hot reload
aurora test	Run test suite
aurora check	Type check without building
aurora lint	Static analysis and style checking
aurora publish --sign	Publish package (requires signing key)
aurora audit	Security audit of all dependencies
aurora update <package>	Update specific dependency
aurora lock	Regenerate lock file

10.4 Package Security

Every package published to the Aurora registry must be cryptographically signed by the author. Packages are distributed as compiled artifacts, not source code. No install scripts execute during dependency resolution.

```
# Generate signing key (stored in hardware if available)
aurora keys generate --hardware

# Publish with signature
aurora publish --sign

# Verify a package
aurora verify moon-http

# Audit dependency tree
```

```
aurora audit --deep
# Output:
#   42 packages scanned
#   0 vulnerabilities found
#   All signatures verified
#   No deprecated packages
```

11. Standard Library

Moon ships with a comprehensive standard library covering common operations. No third-party packages needed for typical web application development.

Module	Purpose	Key Types
moon.core	Primitives, collections, ranges	List, Map, Set, Sequence
moon.io	I/O, streams, buffers	InputStream, OutputStream, Buffer
moon.http	HTTP client and server	HttpClient, Request, Response
moon.json	JSON serialization	JsonEncoder, JsonDecoder
moon.crypto	Cryptographic operations	Hash, Cipher, Signature, Key
moon.time	Date, time, duration	Instant, Duration, Clock, TimeZone
moon.text	String processing, regex	Regex, StringBuilder, Encoding
moon.math	Math operations	BigDecimal, BigInteger, Complex
moon.test	Testing framework	TestSuite, Assert, Mock
moon.concurrent	Concurrency primitives	Flow, Actor, Channel, Mutex
moon.view	UI framework (MoonView)	View, State, Navigator, Style
moon.platform	Platform capabilities	Payment, Notification, Biometric
moon.storage	Persistent storage	Database, KeyValue, FileSystem
moon.net	Networking primitives	Socket, WebSocket, URL
moon.media	Audio, video, image	Camera, Microphone, MediaPlayer

12. Interoperability

Moon is designed as a polyglot compilation target. Developers can write in Moon directly or use supported source languages that compile to Moon's intermediate representation (MIR).

12.1 Supported Source Languages

Language	Status	Compilation Path
Moon (native)	Primary	Moon -> MIR -> Bytecode
Kotlin	Tier 1	Kotlin -> KIR -> MIR -> Bytecode

Swift	Tier 1	Swift -> SIL -> MIR -> Bytecode
Rust	Tier 2	Rust -> LLVM IR -> MIR -> Bytecode
TypeScript	Migration	TS -> AST -> MIR (limited)

12.2 MIR (Moon Intermediate Representation)

MIR is the stable compilation target that all languages emit. It is human-readable and can be written directly. If any upstream language introduces breaking changes, the MIR layer remains stable and all compiled packages continue to function.

```
// MIR is not typically hand-written but is inspectable
// aurora build --emit-mir shows the generated MIR

module main

type User = { id: string, name: string, email: string }

fn main() -> unit {
    let user = User { id: "1", name: "Moon", email: "dev@moon.dev" }
    call print(field user.name)
    ret unit
}
```

12.3 Language Stability Guarantee

MIR is the contract between Moon and developers. The following guarantees apply:

- MIR is semantically versioned and backwards compatible within major versions.
- Compiled MIR bytecode from any Moon version will run on any equal-or-higher runtime version.
- If an upstream source language (Kotlin, Swift, Rust) introduces breaking changes, the compilation bridge is updated on Google’s timeline. Existing compiled packages are unaffected.
- Developers who need maximum stability can write MIR directly as an escape hatch.

13. Compilation Pipeline

13.1 Development Mode

During development, Moon uses an interpreter with hot reload for rapid iteration:

```
aurora dev

# → Moon Dev Server v1.0
# → Watching src/ for changes
# → Running on http://localhost:3000
# → Hot reload enabled
# → Type checking: real-time

# Developer edits a .moon file
# → [0.02s] Incremental recompile: src/views/HomePage.moon
# → [0.01s] Hot reload applied (no page refresh)
```

13.2 Production Build

Production builds use AOT (Ahead of Time) compilation for maximum performance:

```
aurora build --release

# → Moon Compiler v1.0
# → Target: web (MoonView runtime)
# → Compiling 47 source files...
# → Type checking: passed
# → Null safety: verified
# → Capability validation: passed
# → Optimizing MIR...
# → Generating bytecode...
# → Tree shaking: removed 12 unused modules
# → Output: dist/app.moonpkg (842 KB)
# → Build complete in 2.3s
```

13.3 Compilation Stages

Stage	Input	Output	Description
Parse	.moon source	AST	Lexing and parsing to abstract syntax tree
Type Check	AST	Typed AST	Full type inference and null safety verification
Lower	Typed AST	MIR	Convert to Moon

			Intermediate Representation
Optimize	MIR	Optimized MIR	Dead code elimination, inlining, constant folding
Codegen	Optimized MIR	Bytecode	Generate runtime bytecode
Package	Bytecode + assets	.moonpkg	Bundle with manifest and signatures

14. Migration Strategy

The web cannot transition overnight. Moon's migration strategy operates on two tracks running in parallel within the browser.

14.1 Dual Runtime

Chromium will ship with both the legacy web engine (HTML/CSS/JS) and the Moon runtime. Legacy websites continue to work unchanged. New applications can target Moon exclusively. A bridge layer allows Moon applications to embed legacy web content in an iframe-like container for gradual migration.

14.2 Migration Timeline

Phase	Timeline	Milestone
Alpha	2026 Q3	Language spec finalized, compiler prototype, dev tools alpha
Beta	2027 Q1	Aurora registry launch, MoonView framework beta, Chrome Canary integration
Developer Preview	2027 Q3	Public SDK, documentation site, example applications
Chrome Integration	2028 Q1	Moon runtime ships in Chrome stable behind a flag
General Availability	2028 Q3	Flag removed, Moon runtime available to all Chrome users
Legacy Deprecation	2030+	Begin signaling deprecation timeline for legacy JS engine

14.3 TypeScript Migration Tool

A dedicated migration tool will convert TypeScript projects to Moon with high fidelity:

```
aurora migrate --from typescript --source ./src

# → Analyzing 234 TypeScript files...
# → Detected: React components, REST API calls, Redux state
# → Converting types: 98% automated
# → Converting components to MoonView: 85% automated
# → Manual review needed: 12 files (marked with // TODO: MIGRATE)
# → Output: ./src-moon/
```

15. Appendix: Grammar Reference (EBNF)

The following is an abbreviated formal grammar for Moon in Extended Backus-Naur Form:

```
program          = { declaration } ;
declaration      = funDecl | viewDecl | classDecl | enumDecl
                 | interfaceDecl | actorDecl | typeAlias ;

funDecl          = ["suspend"] "fun" identifier
                   [typeParams] "(" [paramList] ")"
                   [":" type] block ;

viewDecl         = "view" identifier "(" [paramList] ")" viewBlock ;

classDecl        = ["data" | "sealed"] "class" identifier
                   [typeParams] ["(" [paramList] ")"]
                   [": " typeList] classBlock ;

actorDecl        = "actor" identifier classBlock ;

enumDecl         = "enum" "class" identifier
                   ["(" paramList ")"] enumBlock ;

navigationDecl   = "navigation" identifier navigationBlock ;

typeParams       = "<" typeParam { "," typeParam } ">" ;
typeParam        = identifier [":" type] ;

type             = identifier [typeArgs] ["?"]
                 | functionType
                 | tupleType ;
functionType     = "(" [typeList] ")" "->" type ;
typeArgs         = "<" type { "," type } ">" ;

statement        = valDecl | varDecl | assignment | expression
                 | ifExpr | whenExpr | forLoop | whileLoop
                 | returnStmt | throwStmt ;

valDecl          = "val" identifier [":" type] "=" expression ;
varDecl          = ["weak" | "unowned"] "var" identifier
                   [":" type] "=" expression ;

whenExpr         = "when" "(" expression ")" "{"
                   { whenEntry } "}" ;
whenEntry        = pattern "->" (expression | block) ;
```

```
lambda            = "{" [paramList "->" ] statements "}" ;
```

—— End of Specification ——